第06堂｜檔案系統運維與傳統系統介面整合

學習目標

完成本堂課後，學員將能夠：

- 建立複雜的檔案自動化管理系統（整理、搬移、重新命名、備份）
- 從 PDF 文件中提取非結構化文字資料並轉化為可用資訊
- 使用桌面 UI 自動化技術，讓 AI 助理能夠操控傳統桌面軟體
- 理解「遺留系統（Legacy System）整合」的概念與實務挑戰

核心概念說明

1. 為什麼「檔案管理與傳統系統整合」是實務中的重要需求？

在實際的企業環境中，有大量未被數位化或 API 化的工作：

常見的企業痛點：

情境	問題
每月收到數百份 PDF 發票	需要手動打開每份、複製金額和日期到 Excel
公司使用 20 年前的進銷存系統	系統無法匯出資料，只能靠人工抄寫
員工桌面資料夾一片混亂	找不到資料，版本管理混亂
掃描文件需要數位化	大量紙本資料未被有效利用

這些場景沒有可用的 API，只能透過「模擬人類操作」的方式自動化。

---

2. 檔案系統操作

Python 提供了豐富的標準函式庫來處理檔案和資料夾：

模組	主要用途
`os`	基礎的作業系統操作（路徑、目錄、環境變數）
`pathlib`	現代化的路徑處理（推薦使用）
`shutil`	檔案複製、搬移、刪除、壓縮
`glob`	依照模式搜尋檔案（支援萬用字元）
`watchdog`	監控資料夾，偵測新檔案的出現

常見的檔案自動化任務：

```
資料夾整理：
    掃描指定資料夾
    依照副檔名分類（.pdf → PDF資料夾、.xlsx → Excel資料夾）
    依照日期分類（今年/去年、月份）
    依照檔案命名規則分類

批次重新命名：
    讀取所有符合條件的檔案
    依照規則生成新的檔名（例如：加上日期前綴）
    重新命名並記錄對照表

自動備份：
    監控重要資料夾的異動
    偵測到新增或修改時，自動複製到備份位置
    保留指定數量的歷史版本
```

---

3. PDF 非結構化資料提取

PDF 是企業中最常見的文件格式，但它的資料不像 Excel 那樣容易讀取。

PDF 的兩種主要類型：

類型	說明	提取方式
文字型 PDF	由電腦直接生成，文字可被搜尋	pdfplumber、pdfminer
掃描型 PDF	紙本文件掃描成 PDF，本質上是圖片	OCR（光學字元識別）工具

常見的 PDF 提取工具：

工具	特色
pdfplumber	精確提取文字與表格結構，適合文字型 PDF
PyMuPDF（fitz）	速度快、功能多，適合大批量處理
Tesseract + pytesseract	開源 OCR 引擎，適合掃描型 PDF
Azure AI Document Intelligence	雲端 AI，準確率高，適合複雜版面

AI 輔助的 PDF 資料提取流程：

```
原始 PDF 檔案
    │
    ▼ pdfplumber 提取文字
原始文字（可能格式混亂）
    │
    ▼ LLM 語意理解與結構化
{
    "廠商": "XX 科技股份有限公司",
    "發票號碼": "AB12345678",
```

```
    "發票日期": "2025-07-15",
    "金額": 52500,
    "稅額": 2500,
    "總計": 55000
}
    |
    ▼ 寫入 Excel 或資料庫
結構化資料
```

---

4. 桌面 UI 自動化（RPA 概念）

RPA（Robotic Process Automation，機器人流程自動化）是一種讓電腦程式模擬「人類操作滑鼠和鍵盤」的技術，讓傳統桌面軟體也能被自動化控制。

為什麼需要 RPA？

許多企業有使用了 10-20 年的系統：

- 老舊 ERP 系統（進銷存、財會）
- 政府平台（需要手動填寫大量欄位）
- 不提供 API 的第三方軟體

這些系統唯一的操作方式就是「人坐在電腦前點選」，RPA 讓機器人來做這件事。

Python UI 自動化工具：

工具	適用場景
pyautogui	基礎滑鼠鍵盤操控，跨平台
pygetwindow	管理視窗（最大化、切換焦點）
uiautomation	Windows 原生 UI 元素識別（推薦用於 Windows 應用程式）

UI 自動化的基本操作：

操作	說明
找到視窗	依視窗標題找到目標應用程式
點擊座標	在螢幕特定位置點擊
輸入文字	模擬鍵盤輸入
截圖比對	找到螢幕上特定的圖示或按鈕
等待元素	等待特定視覺元素出現

> **重要提醒：** UI 自動化比 API 自動化脆弱許多——介面稍有變化（版本更新、螢幕解析度改變）就可能導致腳本失效。應加入足夠的偵錯機制，並定期維護。

---

5. 遺留系統整合的策略

面對無法改造的舊系統，常見的整合策略：

策略	說明	難度
UI 自動化	用機器人模擬人工操作	低
資料庫直接存取	直接讀寫舊系統的資料庫	中（需要 IT 授權）
螢幕截圖 + AI 識別	截取畫面，用 AI 理解並提取資訊	中
中介層（Middleware）	在舊系統外包一層 API	高（需要開發）

實作步驟（逐步教學）

實作一：自動整理桌面資料夾

步驟 1：安裝必要套件

```
pip install pathlib  # Python 標準函式庫，通常無需安裝
pip install watchdog  # 資料夾監控
```

步驟 2：向 GitHub Copilot 描述需求

```
請幫我寫一個 Python 程式，自動整理「下載」資料夾：
1. 掃描「下載」資料夾（路徑從環境變數讀取）
2. 依照以下規則自動分類到子資料夾：
   - PDF 檔案 → 「文件/PDF」資料夾
   - Excel/CSV 檔案 → 「文件/試算表」資料夾
   - 圖片（jpg, png, gif）→「媒體/圖片」資料夾
   - 壓縮檔（zip, rar）→「壓縮檔」資料夾
   - 其他 → 「其他」資料夾
3. 如果目標資料夾不存在，自動建立
4. 搬移後紀錄每個檔案的新舊路徑到 log 檔案
5. 如果遇到同名檔案，在新檔名後面加上日期（不要覆蓋）
```

步驟 3：加入 AI 輔助的分類

```
請修改程式，讓 AI 幫忙處理無法依副檔名判斷的情況：
對於副檔名不明確的檔案，用 AI 根據檔名猜測應該放到哪個資料夾。
例如：「2025Q2銷售分析.dat」→ AI 判斷可能是試算表類型
```

實作二：PDF 發票批次資料提取

步驟 1：安裝 PDF 工具

```
pip install pdfplumber
```

步驟 2：向 GitHub Copilot 描述需求

```
請幫我寫一個 PDF 發票自動化提取程式：
1. 掃描「發票資料夾」內所有的 .pdf 檔案
2. 用 pdfplumber 提取每份 PDF 的文字內容
3. 將提取的文字傳給 OpenAI API，讓 AI 辨識並以 JSON 格式回傳：
   {
     "廠商名稱": "",
     "發票號碼": "",
     "發票日期": "",
     "稅前金額": 0,
     "稅額": 0,
     "總金額": 0
   }
4. 如果 AI 無法確定某個欄位，填入 "UNKNOWN"
5. 將所有提取結果匯整成一份 Excel 報表
6. 標記所有含有 "UNKNOWN" 的列，提示需要人工審核
```

實作三：傳統桌面系統自動化示範

步驟 1：安裝 UI 自動化工具

```
pip install pyautogui
pip install pygetwindow
```

步驟 2：示範：自動操控「記事本」（基礎練習）

```
請幫我寫一個 pyautogui 程式：
1. 開啟 Windows 記事本
2. 等待記事本視窗出現
3. 在記事本中輸入今天的日期和「自動化測試成功」
4. 儲存檔案到指定路徑
```

步驟 3：進階：抓取老舊系統的螢幕資料

透過截圖 + AI Vision 的方式提取傳統系統中的資料：

```
請幫我寫一個程式：
1. 找到名為「XXX 系統」的視窗並切換到前景
2. 對該視窗截圖
```

```
3. 將截圖傳給 OpenAI Vision API，請 AI 提取畫面中的資料欄位
4. 將提取的資料存入 Excel
```

程式碼範例（虛擬碼）

資料夾自動整理

```
載入 pathlib, shutil, datetime 套件

# 設定路徑
來源資料夾 = 路徑("~/Downloads")
目標根資料夾 = 路徑("~/下載-已整理")

# 依副檔名對應目標資料夾
分類規則 = {
    ".pdf": "文件/PDF",
    ".xlsx": "文件/試算表",
    ".csv": "文件/試算表",
    ".jpg": "媒體/圖片",
    ".png": "媒體/圖片",
    ".zip": "壓縮檔",
    ".rar": "壓縮檔",
}

搬移紀錄 = []

# 遍歷所有檔案
對 每個檔案 在 來源資料夾 中（只看檔案，不看子資料夾）:
    副檔名 = 取得副檔名(檔案).轉小寫()

    如果 副檔名 在 分類規則 中:
        目標子資料夾 = 分類規則[副檔名]
    否則:
        目標子資料夾 = "其他"

    目標路徑 = 目標根資料夾 / 目標子資料夾
    建立資料夾(目標路徑, 含父資料夾=真, 若已存在不報錯=真)

    # 處理同名衝突
    最終目標 = 目標路徑 / 檔案.名稱
    如果 最終目標 已存在:
        當前日期 = 取得當前日期字串()
        新檔名 = f"{檔案.主檔名}_{當前日期}{副檔名}"
        最終目標 = 目標路徑 / 新檔名

    # 執行搬移
    搬移(檔案, 最終目標)
    搬移紀錄.加入({"原始路徑": 檔案, "新路徑": 最終目標})

# 寫入 log
```

```
將搬移紀錄寫入("整理紀錄.csv")
顯示 f"完成！共整理 {len(搬移紀錄)} 個檔案"
```

PDF 資料提取

```
載入 pdfplumber, openai, pandas 套件

函式 提取PDF內容(pdf檔案路徑):
    使用 pdfplumber.open(pdf檔案路徑) 作為 pdf:
        全文 = ""
        對 每一頁 在 pdf.pages 中:
            全文 += 每一頁.extract_text()
        回傳 全文

函式 AI解析發票(原始文字):
    提示詞 = f"""
    以下是一份發票的文字內容：
    {原始文字}

    請提取以下欄位並以 JSON 格式回傳，無法判斷的欄位請填 "UNKNOWN"：
    {{
        "廠商名稱": "",
        "發票號碼": "",
        "發票日期": "",
        "稅前金額": 0,
        "稅額": 0,
        "總金額": 0
    }}
    只回傳 JSON，不要說明。
    """
    回應 = 呼叫LLM(提示詞)
    回傳 解析JSON(回應)

# 批次處理
所有結果 = []
對 每個pdf檔案 在 列出資料夾("發票資料夾", "*.pdf") 中:
    顯示 f"處理中：{pdf檔案.名稱}"
    文字 = 提取PDF內容(pdf檔案)
    結果 = AI解析發票(文字)
    結果["原始檔案"] = pdf檔案.名稱
    所有結果.加入(結果)

# 輸出報表
報表 = 建立DataFrame(所有結果)
報表["需人工審查"] = 報表.任何欄位包含("UNKNOWN")
報表.寫入Excel("發票彙整.xlsx")
```

練習題 / 作業

課堂練習

練習 1：資料夾整理規則設計

針對你真實的工作資料夾，設計一套整理規則：

- 你希望依照哪些維度分類（類型、日期、專案名稱）？
- 對於不確定如何分類的檔案，希望 AI 如何處理？
- 整理後的資料夾結構長什麼樣？

練習 2：PDF 提取難度評估

找找看你手邊的 PDF 文件，判斷它是哪種類型：

- 可以複製文字的 → 文字型 PDF（容易處理）
- 無法複製文字的 → 掃描型 PDF（需要 OCR）

並思考：這份 PDF 的格式有多複雜？AI 能夠正確提取嗎？可能的困難點在哪？

課後作業

1. **資料夾整理程式**：用 GitHub Copilot 協助撰寫一個自動整理你電腦某個資料夾的腳本（可以先用測試資料夾）。
2. **PDF 提取實作**：找一份簡單的 PDF 文件（例如報名表或收據），嘗試用 pdfplumber 提取文字，再讓 AI 整理成結構化資料，觀察成功率和可能的問題。

延伸閱讀

檔案系統操作：

- Python pathlib 文件 — https://docs.python.org/3/library/pathlib.html
- watchdog — 資料夾監控套件 — https://python-watchdog.readthedocs.io/

PDF 處理：

- pdfplumber — https://github.com/jsvine/pdfplumber
- PyMuPDF 文件 — https://pymupdf.readthedocs.io/

桌面自動化：

- pyautogui 文件 — https://pyautogui.readthedocs.io/
- 《Automate the Boring Stuff with Python》— 第 18-20 章，桌面自動化相關章節